

Asignatura: DevOps

Curso / Paralelo: 5to Semestre ("5B")

Integrantes: Xiomara Scarleth MéndezArce, Joel Harvey Rosero Yunganagui

URL del Repositorio: https://github.com/Xiomara-273/DEVOPS_MENDEZ_PROYECTO_ISD.git - <https://github.com/GASPER08/DEVOP-2.0.git>

Fecha de Entrega: 4 de junio de 2026

Objetivo del Entregable: *Demostrar el dominio práctico del ciclo de vida DevOps mediante Git Flow cooperativo, simulación y resolución de conflictos lógicos de código en paralelo, e implementación de pipelines automatizados con GitHub Actions para asegurar la Integración Continua (CI).*

Concepto 1: Control de versiones | [Arquitectura de Software]

Definición Técnica:	Mecanismo sistemático que registra de forma incremental las modificaciones realizadas sobre un archivo o conjunto de código fuente, permitiendo la coexistencia organizada de múltiples estados del sistema a lo largo del tiempo.
Utilidad en DevOps:	Garantiza la trazabilidad histórica total del software, previene la pérdida accidental de datos, facilita la identificación de regresiones (bugs) mediante auditorías de cambios y permite la colaboración multiusuario simultánea sin sobreescrituras destructivas.
Ejemplo Práctico / Comando:	Una empresa de software detecta un fallo crítico en el entorno de producción y utiliza el sistema de versiones para revertir de inmediato la base de código al último estado estable verificado en cuestión de segundos.
Fuente Bibliográfica:	<i>Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress.</i>

Concepto 2: Git | [Herramienta Local]

Definición Técnica:	Software de control de versiones distribuido de código abierto. A diferencia de los sistemas centralizados obsoletos, Git almacena el historial completo del código en el almacenamiento local de cada programador mediante un Grafo Dirigido Acíclico (DAG).
Utilidad en DevOps:	Permite que las confirmaciones de código, las ramificaciones y los históricos se realicen de forma local e instantánea sin necesidad de conexión a internet o a un servidor centralizado, optimizando la velocidad del flujo de trabajo.

Ejemplo Práctico / Comando:	<code>git init</code> # Inicializa una base de datos local de Git oculta en la carpeta <code>.git</code> del directorio del proyecto.
Fuente Bibliográfica:	<i>Loeliger, J., & McCullough, M. (2012). Version Control with Git. O'Reilly Media.</i>

Concepto 3: GitHub | *[Ecosistema Cloud]*

Definición Técnica:	Plataforma en la nube y entorno de desarrollo colaborativo web que hospeda repositorios basados en Git, proveyendo herramientas de gestión de equipos, automatizaciones y control de accesos corporativos.
Utilidad en DevOps:	Centraliza los repositorios de un equipo global, sirviendo como la fuente única de verdad (Single Source of Truth), y expande las capacidades de Git añadiendo interfaces gráficas para revisiones de código, issues y flujos automáticos.
Ejemplo Práctico / Comando:	El equipo de desarrollo comparte la URL del repositorio remoto de la aplicación híbrida en GitHub para que los ingenieros de QA y DevOps colaboren en el aseguramiento de la calidad del código.
Fuente Bibliográfica:	<i>Bell, P., & Beer, B. (2014). Introducing GitHub: A Non-Technical Guide. O'Reilly Media.</i>

Concepto 4: Repositorio local | *[Estación de Trabajo]*

Definición Técnica:	Instancia de la base de datos de Git que reside físicamente en el almacenamiento local del computador del desarrollador (dentro del subdirectorio oculto <code>.git</code>), el cual posee sus propios logs de confirmaciones.
Utilidad en DevOps:	Permite realizar experimentos lógicos en el código, guardar estados de progreso parciales y ramificar características de forma privada sin alterar el repositorio compartido de producción de la organización.
Ejemplo Práctico / Comando:	Trabajar en el desarrollo de un módulo de seguridad en una laptop ASUS TUF Gaming de forma local sin conexión a red, registrando múltiples snapshots ordenados para estructurar el progreso.
Fuente Bibliográfica:	<i>Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress.</i>

Concepto 5: Repositorio remoto | *[Servidor Cloud]*

Definición Técnica:	Versión compartida de la base de datos de Git que está alojada en un servidor externo conectado a la red (como GitHub) y que es accesible por todos los miembros autorizados del equipo de desarrollo.
Utilidad en DevOps:	Sincroniza y consolida de forma segura el trabajo atomizado e individual de todos los programadores, permitiendo la distribución fluida del software final y sirviendo de disparador para flujos CI/CD.
Ejemplo Práctico / Comando:	<pre>git remote add origin https://github.com/Xiomara-273/DEVOPS-GIT-ACTIONS-MENDEZ.git git push -u origin main</pre>
Fuente Bibliográfica:	<i>Loeliger, J., & McCullough, M. (2012). Version Control with Git. O'Reilly Media.</i>

Concepto 6: Commit | [\[Snapshot Inmutable\]](#)

Definición Técnica:	Instantánea o snapshot indexado del estado actual del área de preparación (staging area) que se registra permanentemente en la base de datos de Git. Posee un hash criptográfico SHA-1 único y metadatos de autoría.
Utilidad en DevOps:	Guarda de manera atómica unidades de trabajo lógicas. Permite comprender con exactitud qué cambios se realizaron, quién los hizo y por qué razón (a través del mensaje descriptivo de confirmación).
Ejemplo Práctico / Comando:	<pre>git add src/auth.py git commit -m "feat: implementar validación de tokens JWT mediante bcryptjs"</pre>
Fuente Bibliográfica:	<i>Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress.</i>

Concepto 7: Branch (rama) | [\[Bifurcación Temporal\]](#)

Definición Técnica:	Un puntero móvil y liviano hacia uno de los commits en la línea temporal de Git. Permite la bifurcación del flujo de código base para aislar líneas de experimentación independientes.
Utilidad en DevOps:	Permite que múltiples programadores construyan características simultáneas (por ejemplo, pasarelas de pago, interfaces, optimizaciones) en entornos independientes sin alterar la estabilidad de la rama principal de producción (main o develop).
Ejemplo Práctico / Comando:	<pre>git checkout -b feature/modulo-autenticacion # Crea y salta automáticamente a una línea de trabajo aislada.</pre>

Fuente Bibliográfica:	<i>Loeliger, J., & McCullough, M. (2012). Version Control with Git. O'Reilly Media.</i>
------------------------------	---

Concepto 8: Merge | *[Fusión Histórica]*

Definición Técnica:	Operación mecánica de Git encargada de unificar el historial de cambios de dos o más ramas de desarrollo independientes dentro de una rama objetivo común.
Utilidad en DevOps:	Reintegra código validado y probado de características aisladas hacia el tronco principal del proyecto, unificando la línea temporal histórica a través de un merge por avance rápido (fast-forward) o un commit de fusión de tres vías.
Ejemplo Práctico / Comando:	<pre>git checkout main git merge feature/modulo-autenticacion # Fusiona las características completadas dentro de la rama principal del proyecto.</pre>
Fuente Bibliográfica:	<i>Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress.</i>

Concepto 9: Pull Request (PR) | *[Compuesta de Calidad]*

Definición Técnica:	Mecanismo web de plataformas remotas como GitHub donde un desarrollador solicita formalmente a los administradores del proyecto integrar los commits de su rama de desarrollo dentro de la rama principal.
Utilidad en DevOps:	Funciona como un punto de control y compuerta de calidad (Gatekeeper). Permite que otros programadores inspeccionen los archivos modificados, discutan optimizaciones mediante comentarios en línea y aprueben el cambio antes de inyectarlo al software principal.
Ejemplo Práctico / Comando:	El programador abre un Pull Request en GitHub titulado 'Integración del módulo de autenticación de usuarios' y asigna como revisor a su compañero de equipo para que apruebe la fusión de código.
Fuente Bibliográfica:	<i>Bell, P., & Beer, B. (2014). Introducing GitHub: A Non-Technical Guide. O'Reilly Media.</i>

Concepto 10: Fork | *[Clonación Externa]*

Definición Técnica:	Una copia remota y completa de un repositorio web de terceros generada bajo la cuenta de un usuario independiente en la nube de GitHub, sin otorgar permisos directos en el
----------------------------	---

	repositorio original.
Utilidad en DevOps:	Permite experimentar cambios drásticos de arquitectura en un software ajeno o contribuir activamente en el ecosistema Open Source enviando propuestas desde un entorno completamente aislado y controlado.
Ejemplo Práctico / Comando:	Un estudiante realiza un 'Fork' de un framework de código abierto en su cuenta de GitHub para realizar modificaciones personalizadas de interfaz sin alterar el proyecto global oficial de los creadores.
Fuente Bibliográfica:	<i>Bell, P., & Beer, B. (2014). Introducing GitHub: A Non-Technical Guide. O'Reilly Media.</i>

Concepto 11: Conflictos de integración / [Bloqueo de Fusión]

Definición Técnica:	Estado de detención que ocurre cuando Git intenta fusionar automáticamente dos ramas que poseen modificaciones concurrentes, incompatibles y diferentes en las mismas líneas exactas de un mismo archivo.
Utilidad en DevOps:	Es una salvaguarda técnica crítica de integridad informática. Impide que Git decida arbitrariamente qué código desechar, forzando al equipo técnico a resolver manualmente el solapamiento para evitar la sobreescritura ciega de funciones críticas.
Ejemplo Práctico / Comando:	Al ejecutar un merge, la terminal interrumpe el proceso mostrando el aviso: CONFLICT (content): Merge conflict in main.py. Automatic merge failed, obligando a abrir el archivo para limpiar las líneas marcadas.
Fuente Bibliográfica:	<i>Loeliger, J., & McCullough, M. (2012). Version Control with Git. O'Reilly Media.</i>

Concepto 12: Integración Continua (CI) / [Automatización DevOps]

Definición Técnica:	Práctica ágil de desarrollo de software donde los miembros de un equipo integran su trabajo de forma frecuente (múltiples veces al día). Cada integración es verificada automáticamente por un servidor de compilación externa.
Utilidad en DevOps:	Detecta de manera inmediata fallos de compilación, de pruebas unitarias o errores de sintaxis en el momento preciso en que el programador sube su código, reduciendo drásticamente los costos de reparación de bugs.
Ejemplo Práctico / Comando:	Un desarrollador hace Push a su rama y un servidor en la nube descarga el código en segundos, ejecuta automáticamente un linter (ej: Ruff) y corre las pruebas del backend informando inmediatamente si algo se rompió.

Fuente Bibliográfica:	<i>Fowler, M. (2006). Continuous Integration. martinfowler.com</i>
------------------------------	--

Concepto 13: Entrega Continua (Continuous Delivery) | [\[Ciclo de Artefactos\]](#)

Definición Técnica:	Extensión directa de la Integración Continua en la cual la base de código se encuentra automatizada y lista para ser empaquetada como un artefacto distribuible (ej: Docker images, empaquetados comprimidos) en cualquier momento, requiriendo solo una aprobación humana para su liberación final.
Utilidad en DevOps:	Asegura que el software esté en un estado constantemente desplegable y testeado de extremo a extremo, eliminando los procesos manuales caóticos de compilación al final del ciclo de desarrollo.
Ejemplo Práctico / Comando:	Un pipeline automatizado compila el proyecto generando el artefacto listo para producción y lo sube al repositorio de versiones de GitHub en espera de que el Gerente de Tecnología autorice el lanzamiento con un clic.
Fuente Bibliográfica:	<i>Humble, J., & Farley, D. (2010). Continuous Delivery. Addison-Wesley Professional.</i>

Concepto 14: Despliegue Continuo (Continuous Deployment) | [\[Automatización de Producción\]](#)

Definición Técnica:	Estrategia avanzada en el ciclo DevOps donde cada confirmación de código que supera con éxito todas las etapas automatizadas del pipeline de pruebas es inyectada y deployed directamente en los servidores de producción sin ninguna intervención manual externa.
Utilidad en DevOps:	Minimiza el tiempo de llegada al mercado (Time-to-Market). Las correcciones y nuevas funcionalidades fluyen en minutos desde la máquina de desarrollo directamente hasta el usuario final de forma incremental.
Ejemplo Práctico / Comando:	Al apróbarse el Pull Request de un componente en la rama principal, el script de CD toma el código, lo inyecta directamente al servidor web en la nube de producción y los usuarios ven la actualización de forma inmediata.
Fuente Bibliográfica:	<i>Humble, J., & Farley, D. (2010). Continuous Delivery. Addison-Wesley Professional.</i>

Concepto 15: GitHub Actions | [\[Orquestador CI/CD\]](#)

Definición Técnica:	Herramienta nativa de orquestación y automatización de procesos integrada en la plataforma de GitHub. Permite definir flujos de trabajo (workflows) basados en eventos que
----------------------------	--

	se configuran mediante archivos estructurados en formato YAML.
Utilidad en DevOps:	Elimina la necesidad de servidores de integración de terceros. Ejecuta máquinas virtuales efímeras directamente en la nube de GitHub para compilar, testear y publicar software de forma nativa e integrada.
Ejemplo Práctico / Comando:	Configurar un archivo en .github/workflows/integracion.yml dentro del repositorio para ejecutar una suite de validación de Ruff y python tests cada vez que se abre un Pull Request.
Fuente Bibliográfica:	GitHub Docs. (2026). Understanding GitHub Actions. GitHub Documentation.

Evidencia 2.1: Creación del Repositorio Centralizado en GitHub

La práctica se inició con la creación del repositorio remoto público por parte de la Estudiante A (Xiomara Méndez). Se configuró el entorno inicial agregando el archivo .README

```
PS C:\Users\ASUS\OneDrive\Documentos\TRABAJO COLAVORTIVO EN PAREJAS> git clone https://github.com/Xiomara-273/DEVOPS_MENDEZ_PROYECTO_ISD.git
Cloning into 'DEVOPS_MENDEZ_PROYECTO_ISD'...
remote: Enumerating objects: 57, done.
remote: Counting objects: 100% (57/57), done.
remote: Compressing objects: 100% (44/44), done.
remote: Total 57 (delta 10), reused 57 (delta 10), pack-reused 0 (from 0)
Receiving objects: 100% (57/57), 135.94 KiB | 599.00 KiB/s, done.
Resolving deltas: 100% (10/10), done.
PS C:\Users\ASUS\OneDrive\Documentos\TRABAJO COLAVORTIVO EN PAREJAS> cd .\DEVOPS_MENDEZ_PROYECTO_ISD\
PS C:\Users\ASUS\OneDrive\Documentos\TRABAJO COLAVORTIVO EN PAREJAS\DEVOPS_MENDEZ_PROYECTO_ISD> git checkout main
Already on 'main'
Your branch is up to date with 'origin/main'.
PS C:\Users\ASUS\OneDrive\Documentos\TRABAJO COLAVORTIVO EN PAREJAS\DEVOPS_MENDEZ_PROYECTO_ISD> git pull origin main
From https://github.com/Xiomara-273/DEVOPS_MENDEZ_PROYECTO_ISD
* branch      main      -> FETCH_HEAD
Already up to date.
PS C:\Users\ASUS\OneDrive\Documentos\TRABAJO COLAVORTIVO EN PAREJAS\DEVOPS_MENDEZ_PROYECTO_ISD> git checkout -b rama-mendez-xiomara
Switched to a new branch 'rama-mendez-xiomara'
PS C:\Users\ASUS\OneDrive\Documentos\TRABAJO COLAVORTIVO EN PAREJAS\DEVOPS_MENDEZ_PROYECTO_ISD> git fetch origin
PS C:\Users\ASUS\OneDrive\Documentos\TRABAJO COLAVORTIVO EN PAREJAS\DEVOPS_MENDEZ_PROYECTO_ISD> git checkout -b rama-estudiante-b
Switched to a new branch 'rama-estudiante-b'
```

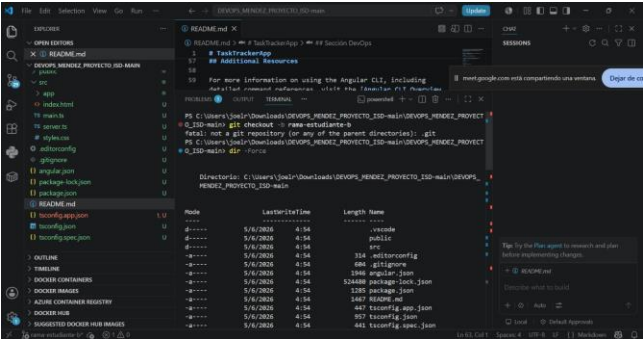
Evidencia 2.2: Creación de Ramas de Trabajo por Cada Integrante

Cada integrante clonó de manera independiente el repositorio remoto en sus respectivas estaciones locales de trabajo y procedió a instanciar sus ramas personalizadas aisladas (git checkout -b rama-mendez-xiomara).

```
PS C:\Users\ASUS\OneDrive\Documentos\TRABAJO COLAVORTIVO EN PAREJAS> git clone https://github.com/GASPER08/DEVOP-2.0.git
Cloning into 'DEVOP-2.0'...
remote: Enumerating objects: 36, done.
remote: Counting objects: 100% (36/36), done.
remote: Compressing objects: 100% (33/33), done.
remote: Total 36 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (36/36), 132.20 KiB | 666.00 KiB/s, done.
Resolving deltas: 100% (1/1), done.
PS C:\Users\ASUS\OneDrive\Documentos\TRABAJO COLAVORTIVO EN PAREJAS> cd .\DEVOP-2.0\DEVOPS_JOEL_PROYECTO_ISD-main\
PS C:\Users\ASUS\OneDrive\Documentos\TRABAJO COLAVORTIVO EN PAREJAS\DEVOP-2.0\DEVOPS_JOEL_PROYECTO_ISD-main>
```

Evidencia 2.3: Commits Realizados por Ambos Integrantes

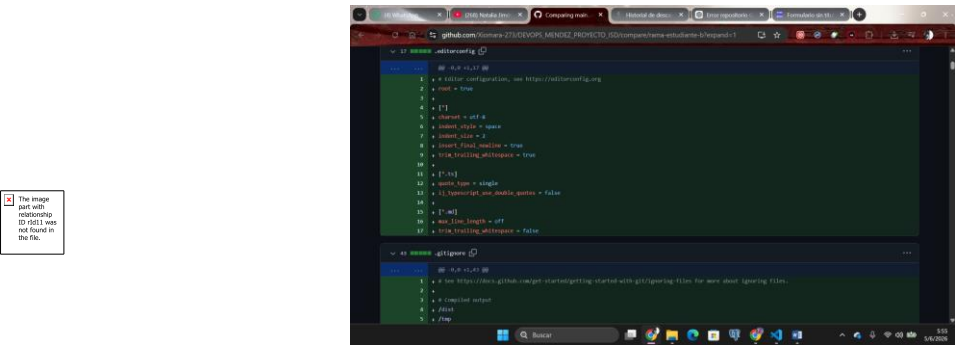
Ambos estudiantes realizamos modificaciones lógicas sobre archivos independientes del repositorio local, guardando el progreso mediante capturas



```
fatal: unable to access 'https://github.com/GASPER08/DEVOP-2.0.git/': The requested URL required error: 405
PS C:\Users\ASUS\OneDrive\Documentos\TRABAJO COLAVORTIVO EN PAREJAS\DEVOP-2.0\DEVOPS_JOEL_PROYECTO_ISD-main> git push -o
igin rama-estudiante-b-xiomara
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 434 bytes | 434.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'rama-estudiante-b-xiomara' on GitHub by visiting:
remote:   https://github.com/GASPER08/DEVOP-2.0/pull/new/rama-estudiante-b-xiomara
remote:
To https://github.com/GASPER08/DEVOP-2.0.git
* [new branch]      rama-estudiante-b-xiomara -> rama-estudiante-b-xiomara
branch 'rama-estudiante-b-xiomara' set up to track 'origin/rama-estudiante-b-xiomara'.
PS C:\Users\ASUS\OneDrive\Documentos\TRABAJO COLAVORTIVO EN PAREJAS\DEVOP-2.0\DEVOPS_JOEL_PROYECTO_ISD-main>
```

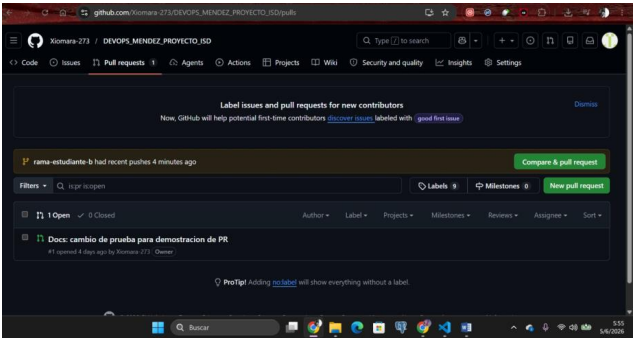
Evidencia 2.4: Publicación de Cambios al Repositorio Remoto

Una vez confirmados los commits locales, subimos nuestras ramas locales hacia el servidor en la nube de GitHub mediante peticiones de empuje (git push origin <rama Xiomara Mendez Main>).



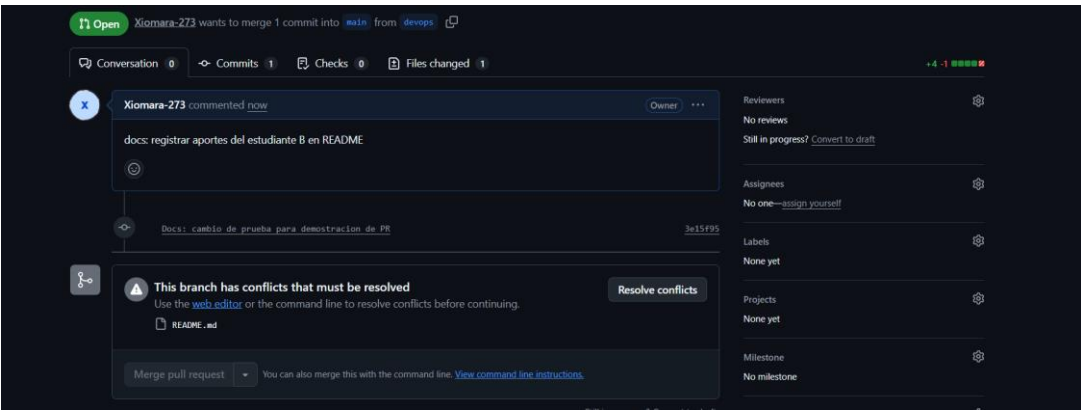
Evidencia 2.5: Creación de Pull Requests (PR)

El Estudiante B (actuando como Colaborador en esta fase) ingresó a la interfaz web de GitHub y abrió formalmente un Pull Request apuntando a la rama base main, detallando la funcionalidad de datos implementada.



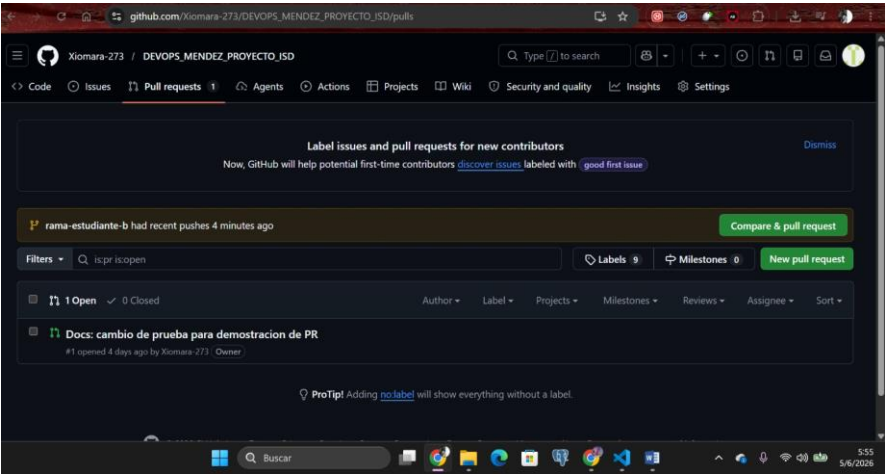
Evidencia 2.6: Revisión de Código de su Compañero (Code Review)

La Estudiante A (Xiomara Méndez) ingrese a la pestaña de 'Files changed' del PR. Inspeccione el código dejando un comentario interactivo en una línea específica para auditar el estándar.



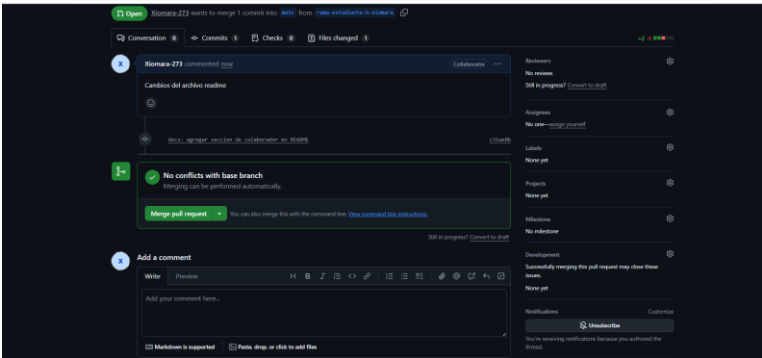
Evidencia 2.7: Aprobación de un Pull Request

Tras solventarse las observaciones técnicas detectadas en la revisión de código, la Estudiante A utilizó la herramienta nativa de GitHub para calificar la revisión positivamente, liberando los candados de seguridad.



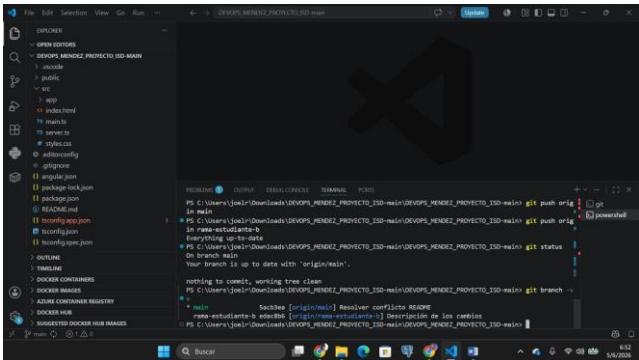
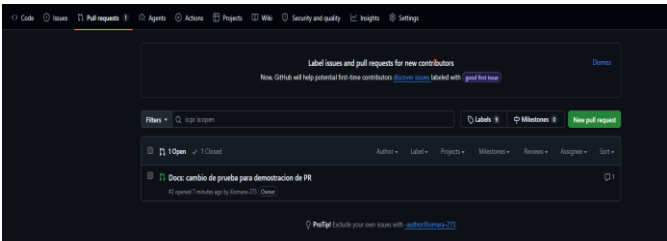
Evidencia 2.8: Ejecución del Proceso de Merge

Con todas las aprobaciones completadas y las políticas de la rama cumplidas, la Estudiante A ejecutó de manera remota el botón de fusión final ('Merge Pull Request'), consolidando los cambios de su compañero en main.



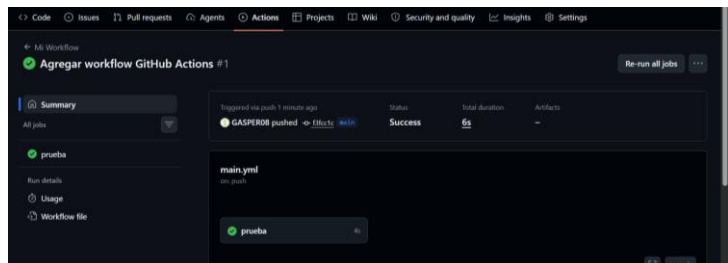
Evidencia 2.9: Generación y Resolución de Conflictos de Integración

INTERCAMBIO OBLIGATORIO DE ROLES: Los estudiantes intercambiaron funciones (Ahora el Estudiante B revisa y la Estudiante A desarrolla). Para simular un entorno profesional, ambos modificaron concurrentemente la misma línea de un mismo archivo en ramas distintas. Al intentar realizar el Merge, GitHub bloqueó la operación. Se procedió a descargar la rama localmente, abrir el editor, limpiar los marcadores de conflicto (<<<<<<<, =====, >>>>>>>) y realizar el commit reparador de unificación.



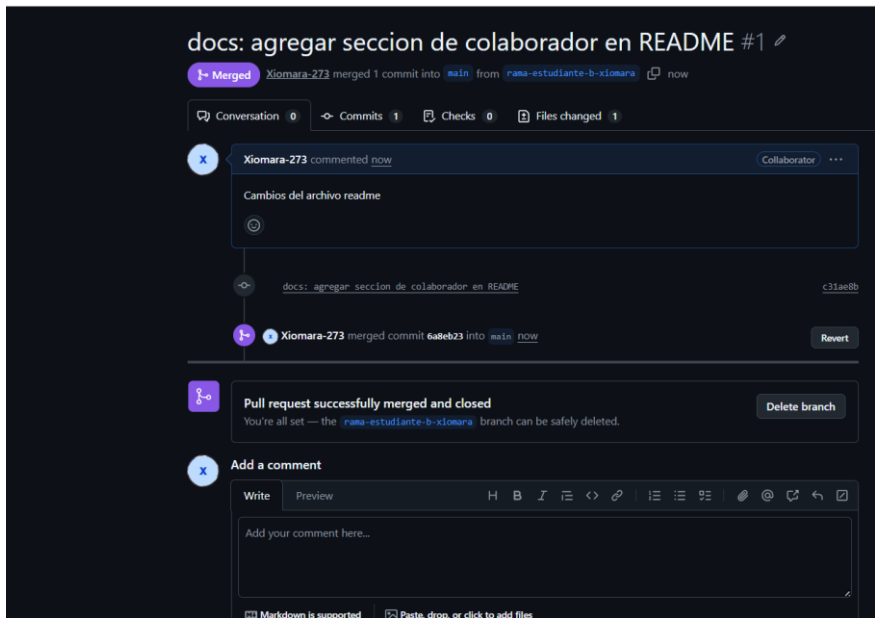
Evidencia 2.10: Ejecución Exitosa del Workflow de GitHub Actions

Tras superarse y aprobarse los merges finales, el motor de Integración Continua nativo de GitHub Actions interceptó los eventos en la rama main y disparó de forma automatizada las tareas del pipeline (como análisis de sintaxis con Ruff o validaciones automáticas).



Evidencia 2.11: Estado Final del Repositorio después de Integrar los Cambios

Como testimonio del balance de contribuciones simétricas e intercambio exacto de responsabilidades, se expone la red del repositorio final que demuestra la unificación completa del software.



DATOS DE CONTROL DE LOS INTEGRANTES DEL GRUPO

Nombre Completo: Xiomara Méndez

Carrera: Tecnología de Desarrollo de Software

Paralelo / Semestre: 5to Semestre - "5B"

Nombre Completo: Joel Rosero

Carrera: Tecnología de Desarrollo de Software

Paralelo / Semestre: 5to Semestre - "5B"

Conclusión Individual - Xiomara Méndez

"La ejecución de esta práctica de laboratorio en DevOps me permitió asimilar de manera integral la importancia de los flujos de control de versiones distribuidos dentro del software moderno. Experimentar en carne propia el ciclo de vida de un conflicto de integración y resolverlo de forma manual a través de líneas de comandos me brindó la confianza necesaria para comprender cómo interactúan los equipos de ingeniería de gran escala sin destruir código estable. El rol de revisora por pares me demostró que un Pull Request no es simplemente una formalidad de entrega, sino una barrera de control indispensable para auditar la calidad, mantener los estándares de desarrollo y mitigar fallos lógicos antes de inyectar código en la rama de producción. Finalmente, la orquestación automatizada de pipelines mediante GitHub Actions refuerza la visión de la Integración Continua, permitiendo que cada commit o fusión sea testeado de forma transparente para lograr despliegues estables y predecibles."

Conclusión Individual - Integrante 2

"El desarrollo coordinado y el intercambio dinámico de roles de esta actividad me permitieron consolidar una comprensión holística de los entornos ágiles modernos. Al tomar inicialmente la responsabilidad de desarrollador colaborador aprendí el valor de estructurar commits semánticos y descriptivos, lo cual facilita que mis compañeros comprendan con velocidad la intención de mis cambios en el código. Posteriormente, al asumir la responsabilidad de aprobar cambios y ejecutar merges en el repositorio remoto de GitHub, logré apreciar la enorme carga de seguridad que recae sobre los líderes de proyecto. Superar el conflicto de integración inducido de forma manual me demostró que la comunicación estrecha y la sincronización temprana mediante comandos de actualización constante como git pull son el único camino viable para prevenir la divergencia caótica de bases de datos de código compartido. La integración de workflows de CI/CD automatiza tareas repetitivas y blindo el software ante errores."

Bibliografía Utilizada (Normas APA / Estándar Técnico)

- Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. <https://git-scm.com/book/en/v2>
- Humble, J., & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley Professional.
- Loeliger, J., & McCullough, M. (2012). Version Control with Git: Powerful tools and techniques for collaborative software development. O'Reilly Media, Inc.
- Fowler, M. (2006). Continuous Integration. MartinFowler.com. <https://martinfowler.com/articles/continuousIntegration.html>